

University Of Engineering and Technology, Lahore
Department of Cyber Security

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Student Name: Hafiz Obaid Ahmed	Roll No: 2026(s)-CYS-47
Semester: Spring 2026	Session: 2026(s)
Lab/Project/Assignment #: 14	CLOs to be covered: CLO2
Lab Title: OS Library and if __name__ == "__main__"	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no

			understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 14: OS Library and if __name__ == "__main__"

Lab Goals/Objectives:

- Understand the purpose of the special variable `__name__` in a Python script.
- Learn how the `if __name__ == "__main__":` construct controls which code runs when a file is executed directly versus imported as a module.
- Practice defining multiple functions in a single file and controlling their execution.

Equipment Required:

- Personal Computer / Laptop
- Python 3.x
- Text Editor / IDE (PyCharm)

Theory:

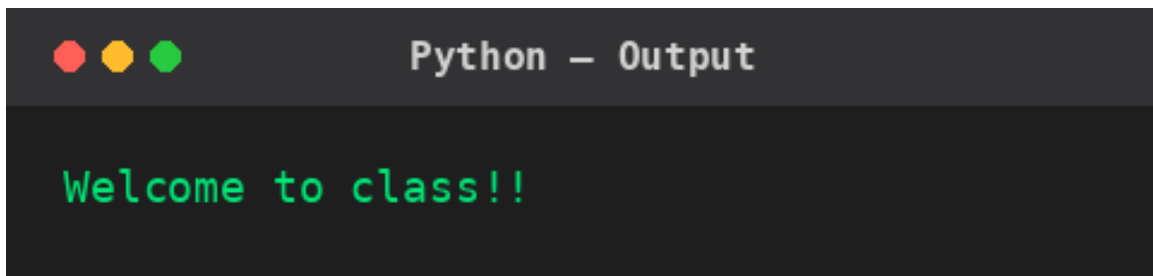
Every Python module has a built-in variable called `__name__`. When a file is run directly, Python sets `__name__` to `"__main__"`; when the same file is imported into another script, `__name__` is instead set to the module's name. The construct `if __name__ == "__main__":` is used to write code that should only execute when the file is run directly, and not when it is imported elsewhere — this is especially useful for keeping helper functions in a file reusable while still allowing the file to be tested or run standalone. This is closely related to Python's `os` library, which is commonly used alongside this pattern to write scripts that behave differently depending on how they are invoked.

Lab Work:

Task 1:

```
def display_message():
    print("Welcome to class!!")
def display_message_two():
    print("Welcome to class2!!")
if __name__ == "__main__":
    display_message()
```

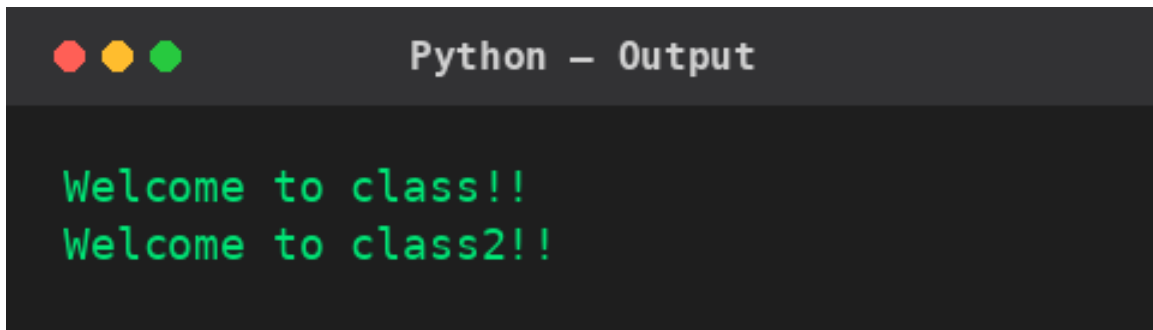
Result:

A terminal window with a dark background and a title bar that says "Python - Output". The title bar has three colored window control buttons (red, yellow, green) on the left. The terminal displays the text "Welcome to class!!" in a green monospaced font.

Task 2:

```
def show_intro():  
    print("Welcome to class!!")  
def show_intro_two():  
    print("Welcome to class2!!")  
if __name__ == "__main__":  
    show_intro()  
    show_intro_two()
```

Result:

A terminal window with a dark background and a title bar that says "Python - Output". The title bar has three colored window control buttons (red, yellow, green) on the left. The terminal displays two lines of text in a green monospaced font: "Welcome to class!!" followed by "Welcome to class2!!" on the next line.

Conclusions:

This lab demonstrated the use of the `if __name__ == "__main__":` construct to control which functions execute when a Python file is run directly, using two simple scripts each defining multiple functions and selectively calling them from the main guard.